

Сайты:

<https://reqres.in>

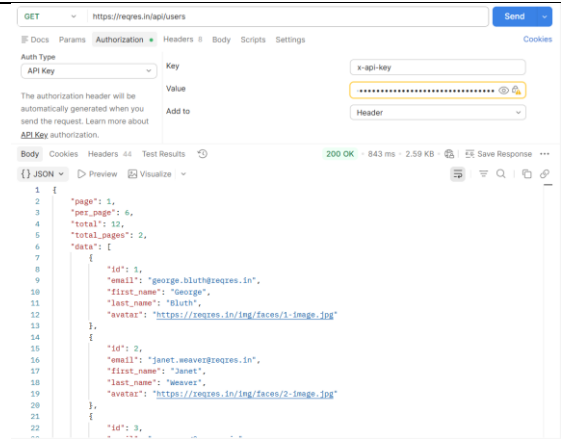
<https://restful-booker.herokuapp.com>

API Checklist (<https://reqres.in>)

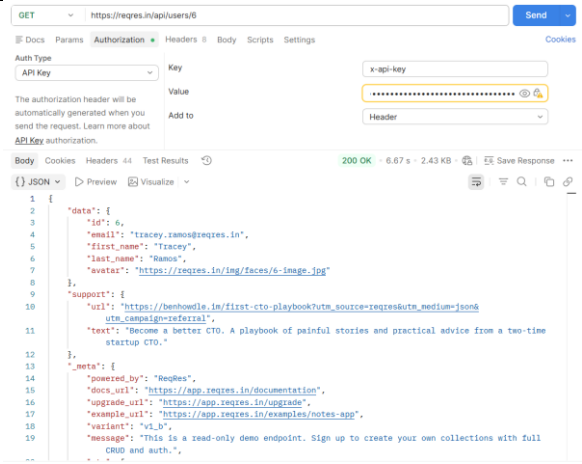
ID	Endpoint	Check Item	Priority	Status
API-001	/users	Проверить GET users	High	Passed
API-002	/users/6	Проверить GET single user	High	Passed
API-003	/users/999	Проверить response для несуществующего user	High	Passed
API-004	/users	Проверить создание пользователя	High	Passed
API-005	/users	Проверить POST с пустым body	High	Failed
API-006	/users	Проверить invalid JSON	Medium	Passed

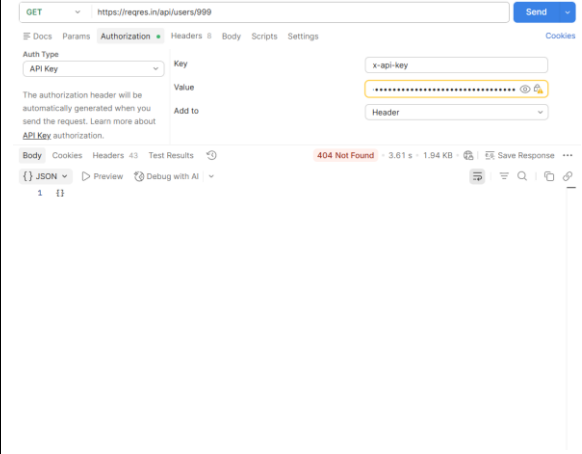
API Checklist (<https://restful-booker.herokuapp.com>)

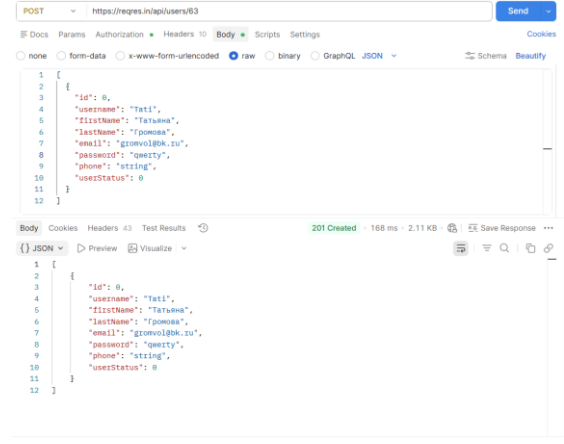
ID	Endpoint	Check Item	Priority	Status
API-001	/ booking	Проверить GET booking	High	Passed
API-002	/ booking /1	Проверить GET single booking	High	Passed
API-003	/ booking /12345	Проверить response для несуществующего booking	High	Passed
API-004	/ booking	Проверить создание booking	High	Passed
API-005	/ booking	Проверить POST с пустым body	High	Failed
API-006	/ booking	Проверить invalid JSON	Medium	Passed

Test Case ID	Test1
Title	Проверка GET users (запрос информации о пользователях)
Preconditions	Postman открыт
Method	GET
Headers	x-api-key
URL	https://reqres.in/api/users
Steps	1. На Workspace создать New collection 2. Add request 3. Отправить GET/users
Expected Result	Status 200, JSON с пользователями
Attachments	

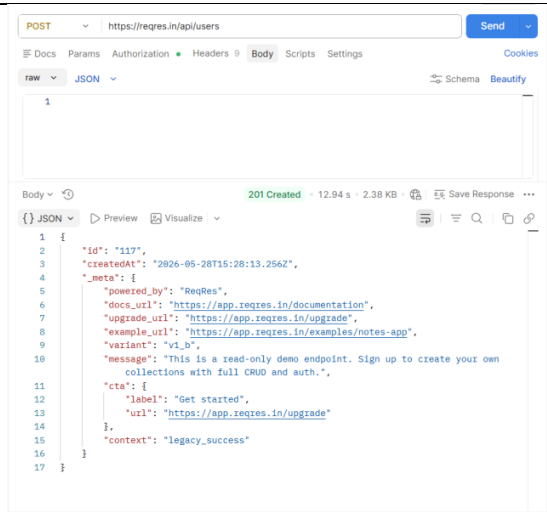
Test Case ID	Test2
Title	Проверка GET single user (запрос информации об одиночном пользователе)
Preconditions	Postman открыт
Method	GET
Headers	x-api-key
URL	https://reqres.in/api/users/6
Steps	1. Add request 2. Отправить GET/users/6

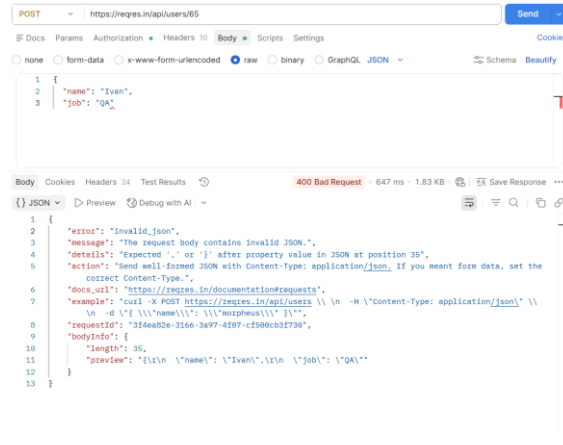
Expected Result	Status 200, JSON с пользователем
Attachments	 The screenshot shows a Postman interface for a GET request to <code>https://reqres.in/api/users/6</code>. The Authorization tab is active, showing an API Key of <code>x-api-key</code> with a masked value. The response is a 200 OK status with a response time of 6.67 s and a body size of 2.43 KB. The JSON response is as follows: <pre>{ "data": { "id": 6, "email": "tracey.ramos@reqres.in", "first_name": "Tracey", "last_name": "Ramos", "avatar": "https://reqres.in/img/faces/6-image.jpg" }, "support": { "url": "https://benhowdle.im/first-cto-playbook?utm_source=reqres&utm_medium=json&utm_campaign=referral", "text": "Become a better CTO. A playbook of painful stories and practical advice from a two-time startup CTO." }, "_meta": { "powered_by": "ReqRes", "docs_url": "https://app.reqres.in/documentation", "upgrade_url": "https://app.reqres.in/upgrade", "example_url": "https://app.reqres.in/examples/notes-app", "variant": "v1.b", "message": "This is a read-only demo endpoint. Sign up to create your own collections with full CRUD and auth." } }</pre>

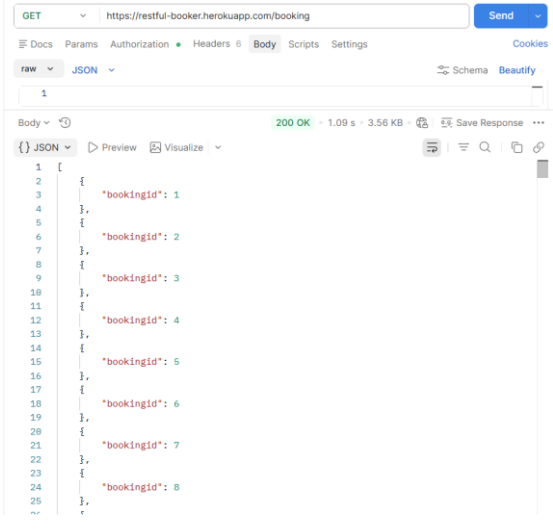
Test Case ID	Test3
Title	Проверка response для несуществующего user
Preconditions	Postman открыт
Method	GET
Headers	x-api-key
URL	https://reqres.in/api/users/999
Steps	1. Add request 2. Отправить GET/users/999
Expected Result	Status 404 Not Found
Attachments	 The screenshot shows a Postman interface for a GET request to <code>https://reqres.in/api/users/999</code>. The Authorization tab is active, showing an API Key of <code>x-api-key</code> with a masked value. The response is a 404 Not Found status with a response time of 3.61 s and a body size of 1.94 KB. The JSON response is as follows: <pre>{ "error": "Not Found" }</pre>

Test Case ID	Test4
Title	Проверка создания пользователя
Preconditions	Postman открыт
Method	POST
Headers	x-api-key
URL	https://reqres.in/api/users
Steps	1. Add request 2. Add body 3. Отправить POST/users
Expected Result	Status 201 Created
Attachments	 The screenshot shows the Postman interface for a POST request to <code>https://reqres.in/api/users/83</code>. The request body is a JSON object: <code>{ "id": 0, "username": "Tati", "firstName": "Татьяна", "lastName": "Громова", "email": "grovol@bk.ru", "password": "qwerty", "phone": "string", "userStatus": 0 }</code>. The response is <code>201 Created</code> with a status bar indicating <code>168 ms - 2.11 KB</code>. The response body is the same JSON object as the request body.

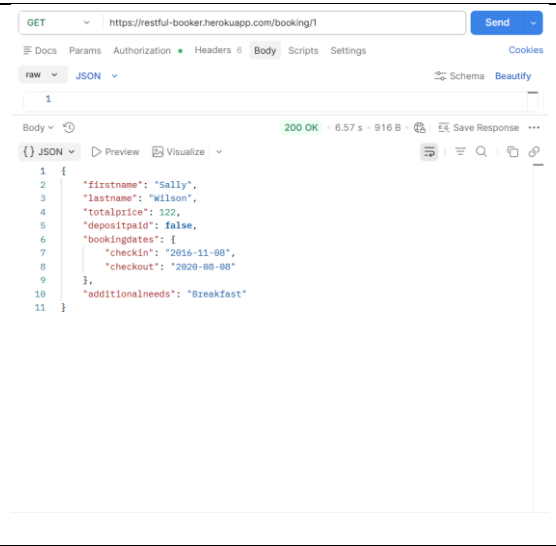
Test Case ID	Test5
Title	Проверка POST с пустым body
Preconditions	Postman открыт
Method	POST
Headers	x-api-key
URL	https://reqres.in/api/users
Steps	1. Add request 2. Add empty body 3. Отправить POST/users
Expected Result	Status 400 Bad Request

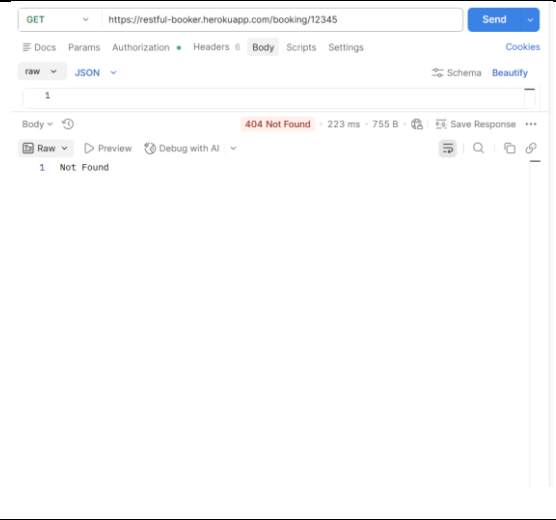
Attachments	 The screenshot shows a Postman interface with a POST request to <code>https://reqres.in/api/users</code>. The response is a JSON object with the following structure: <pre>1 { 2 "id": "117", 3 "createdAt": "2026-05-28T15:28:13.256Z", 4 "_meta": { 5 "powered_by": "ReqRes", 6 "docs_url": "https://app.reqres.in/documentation", 7 "upgrade_url": "https://app.reqres.in/upgrade", 8 "example_url": "https://app.reqres.in/examples/notes-app", 9 "variant": "v1.0", 10 "message": "This is a read-only demo endpoint. Sign up to create your own collections with full CRUD and auth.", 11 "cta": { 12 "label": "Get started", 13 "url": "https://app.reqres.in/upgrade" 14 }, 15 "context": "legacy_success" 16 } 17 }</pre>
-------------	--

Test Case ID	Test6
Title	Проверить invalid JSON
Preconditions	Postman открыт
Method	POST
Headers	x-api-key
URL	https://reqres.in/api/users
Steps	1. Add request 2. Добавить invalid JSON 3. Отправить POST/users
Expected Result	Status 400 Bad Request
Attachments	 The screenshot shows a Postman interface with a POST request to <code>https://reqres.in/api/users/65</code>. The request body is invalid JSON: <code>{ "name": "Ivan", "job": "QA"</code>. The response is a 400 Bad Request error with the following details: <pre>1 { 2 "error": "invalid_json", 3 "message": "The request body contains invalid JSON.", 4 "details": "Expected ',' or ':' after property value in JSON at position 35", 5 "action": "Send well-formed JSON with Content-Type: application/json. If you meant form data, set the correct Content-Type.", 6 "docs_url": "https://reqres.in/documentation#requests", 7 "example": "curl -X POST https://reqres.in/api/users \\n -H 'Content-Type: application/json' \\n -d '{ \\\"name\\\": \\\"morpheus\\\" }'", 8 "requestId": "3f6ea82e-3166-3a97-4597-cf598cb32f30", 9 "bodyInfo": { 10 "length": 36, 11 "preview": "f{r\n \"name\": \"Ivan\",l\n \"job\": \"QA\""} 12 } 13 }</pre>

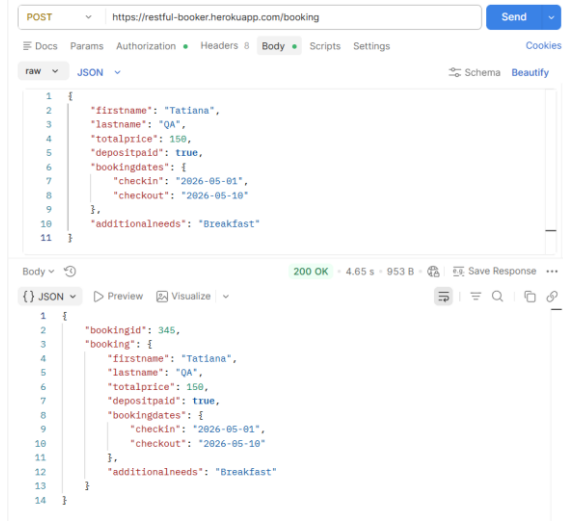
Test Case ID	Test1
Title	Проверка получения списка booking
Preconditions	Postman открыт
Method	GET
URL	https://restful-booker.herokuapp.com/booking
Steps	1. На Workspace создать New collection 2. Add request 3. Отправить GET/ booking
Expected Result	Status 200 OK
Attachments	

Test Case ID	Test2
Title	Проверка получения booking по ID
Preconditions	Postman открыт
Method	GET
URL	https://restful-booker.herokuapp.com/booking/1
Steps	1. Add request 2. Отправить GET/ booking/1
Expected Result	Status 200 OK

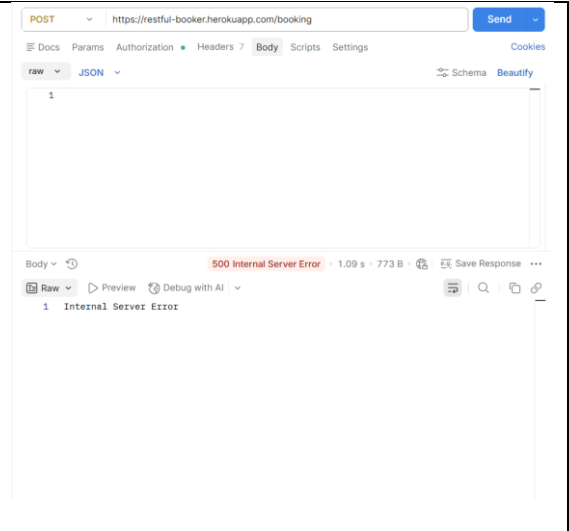
Attachments	 GET https://restful-booker.herokuapp.com/booking/1 200 OK · 6.57 s · 916 B <pre>{ "firstname": "Sally", "lastname": "Wilson", "totalprice": 122, "depositpaid": false, "bookingdates": { "checkin": "2016-11-08", "checkout": "2016-08-08" }, "additionalneeds": "Breakfast" }</pre>
-------------	--

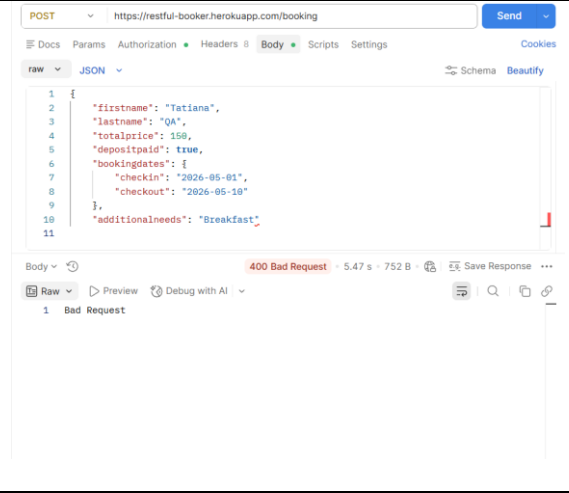
Test Case ID	Test3
Title	Проверка response для несуществующего booking
Preconditions	Postman открыт
Method	GET
URL	https://restful-booker.herokuapp.com/booking/12345
Steps	1. Add request 2. Отправить GET/ booking/12345
Expected Result	Status 404 Not Found
Attachments	 GET https://restful-booker.herokuapp.com/booking/12345 404 Not Found · 223 ms · 755 B 1 Not Found

Test Case ID	Test4
--------------	-------

Title	Проверка создания booking
Preconditions	Postman открыт
Method	POST
URL	https://restful-booker.herokuapp.com/booking
Steps	1. Add request 2. Add body 3. Отправить POST / booking
Expected Result	Status 200 OK
Attachments	 The screenshot shows the Postman interface for a POST request to the URL <code>https://restful-booker.herokuapp.com/booking</code>. The request body is a JSON object with the following structure: <pre> 1 { 2 "firstname": "Tatiana", 3 "lastname": "QA", 4 "totalprice": 150, 5 "depositpaid": true, 6 "bookingdates": { 7 "checkin": "2026-05-01", 8 "checkout": "2026-05-10" 9 }, 10 "additionalneeds": "Breakfast" 11 } </pre> The response is a 200 OK status with a response time of 4.65 s and a body size of 953 B. The response body is a JSON object with the following structure: <pre> 1 { 2 "bookingid": 345, 3 "booking": { 4 "firstname": "Tatiana", 5 "lastname": "QA", 6 "totalprice": 150, 7 "depositpaid": true, 8 "bookingdates": { 9 "checkin": "2026-05-01", 10 "checkout": "2026-05-10" 11 }, 12 "additionalneeds": "Breakfast" 13 } 14 } </pre>

Test Case ID	Test5
Title	Проверка POST с пустым body
Preconditions	Postman открыт
Method	POST
URL	https://restful-booker.herokuapp.com/booking
Steps	1. Add request 2. Add empty body 3. Отправить POST / booking
Expected Result	Status 400 Bad Request

Attachments	
-------------	---

Test Case ID	Test6
Title	Проверить invalid JSON
Preconditions	Postman открыт
Method	POST
URL	https://restful-booker.herokuapp.com/booking
Steps	1. Добавить запрос (Add request) 2. Добавить invalid JSON 3. Отправить POST/ booking
Expected Result	Status 400 Bad Request
Attachments	

Bug Report

Bug ID

BUG-001

Название

POST /api/users создает пользователя с пустым body вместо возврата ошибки валидации

Проект

ReqRes API Testing

Окружение

- API: <https://reqres.in>
- Инструмент: Postman
- Метод: POST
- Endpoint: /api/users

Severity
Medium

Priority
Medium

Preconditions

- Postman открыт
- API key добавлен в Headers

Шаги воспроизведения

1. Открыть Postman
2. Создать POST запрос
3. Указать URL:
<https://reqres.in/api/users>
4. Добавить Header:
x-api-key: <API_KEY>
5. Оставить Body пустым
6. Нажать Send

Ожидаемый результат

Сервер должен вернуть:

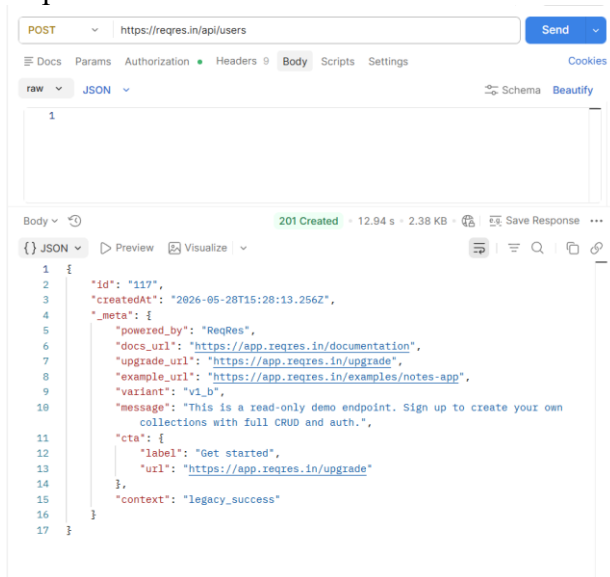
- Status 400 Bad Request
- Сообщение об ошибке валидации
- Пользователь не должен быть создан

Фактический результат

Сервер возвращает успешный ответ и создает пользователя несмотря на пустой request body.

Вложение

Скриншот ответа из Postman



Bug Report

Bug ID
BUG-002

Название

POST /booking возвращает 500 Internal Server Error при пустом request body вместо ошибки валидации

Проект

Restful Booker API Testing

Окружение

- API: <https://restful-booker.herokuapp.com>
- Инструмент: Postman
- Метод: POST
- Endpoint: /booking

Severity

High

Priority

Medium

Preconditions

Postman открыт

Шаги воспроизведения

1. Открыть Postman
2. Создать POST запрос
3. Указать URL:
<https://restful-booker.herokuapp.com/booking>
4. Оставить Body пустым
5. Нажать Send

Ожидаемый результат

Сервер должен вернуть:

- Status 400 Bad Request
- Сообщение об ошибке валидации
- Booking не должен быть создан

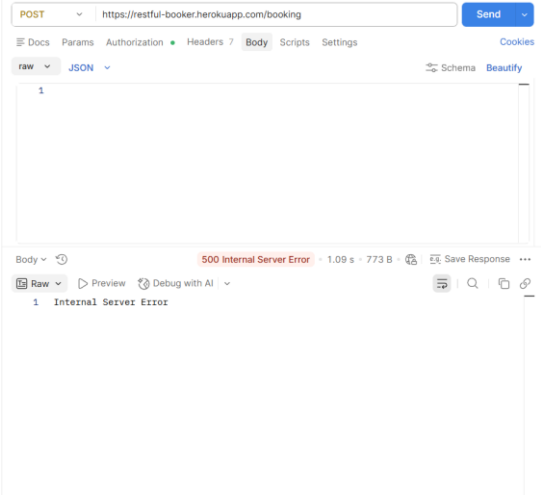
Фактический результат

Сервер возвращает:

- Status 500 Internal Server Error
- Backend не обрабатывает пустой request body корректно.

Вложение

Скриншот ответа из Postman



API Test Report

Проект

API QA Testing Practice Project

Тестируемые API

1. <https://reqres.in>
2. <https://restful-booker.herokuapp.com>

Инструменты

- Postman
- Swagger/OpenAPI
- JSON
- REST API

Цель тестирования

Проверка корректности работы REST API:

- GET requests
- POST requests
- Validation
- Error handling
- Negative testing
- Response codes

Объем тестирования

ReqRes API

Проверены:

- Получение списка пользователей
- Получение пользователя по ID
- Проверка несуществующего пользователя
- Создание пользователя
- POST запрос с пустым body
- Invalid JSON

Restful Booker API

Проверены:

- Получение списка booking
- Получение booking по ID
- Проверка несуществующего booking
- Создание booking
- POST запрос с пустым body
- Invalid JSON

Выполненные тесты

API	Количество тестов
ReqRes	6
Restful Booker	6
Всего	12

Результаты тестирования

Статус Количество

Passed 10

Failed 2

Найденные дефекты

BUG-001

POST /api/users создает пользователя с пустым body вместо возврата validation error.

Severity: Medium

BUG-002

POST /booking возвращает 500 Internal Server Error при пустом body вместо validation error.

Severity: High

Основные проверки

В ходе тестирования были проверены:

- Status Codes
- JSON response
- Response body
- Negative scenarios
- Invalid JSON handling
- Empty body validation
- Not Found responses (404)

Вывод

API testing выполнено частично успешно. Обнаружены 2 дефекта.

В ходе тестирования были обнаружены проблемы обработки некорректных запросов:

- отсутствие корректной validation при пустом body;

- некорректная обработка server-side ошибок.

Проект позволил отработать:

- работу с REST API;
- тестирование через Postman;
- составление test cases;
- negative testing;
- оформление bug reports;
- анализ response codes и response body.